OSAM — Operating State Alignment Module

Parent Standard: Operational Reality Standard
Category: Governance & Enforcement
Subcategory: Operating State Alignment
Type: Operational Reality Architecture Module
Version: 1.0
Status: Canonical · Open Module
Effective Date: 13 May 2026
Compatibility: OOF Methodology OS · Operational Reality Standard · Structured Reality Standard · Runtime Integrity Standard · INTEGROS · MTVF · UCL · ORGS · EVIP
Authority: OOF
Protection: MIP — Methodological Intellectual Property
Canonical Language: English

Canonical Definition

Operating State Alignment Module defines the structural conditions under which a system's declared operating state and its actual live operating state remain continuously aligned strongly enough for the system to remain operationally real during execution.

A system satisfies OSAM only if:

- the declared operating state is explicitly defined
- the actual operating state remains identifiable during runtime
- divergence between declared and actual state can be detected
- operational alignment remains reviewable across time and state change
- the system does not continue claiming one operating condition while functioning under another

A system that operates under one live state while declaring another does not satisfy OSAM.

Module Function

OSAM defines the alignment layer of operational reality governance.

It ensures that live operation does not drift away from the state the system claims to occupy.

The module applies wherever a system must remain aligned between:

- declared runtime role
- actual runtime behavior
- active control conditions
- operational boundary state
- permission status
- execution mode
- consequence-bearing system function

Its function is not merely to record activity.

Its function is to determine whether the system is still operating
in the same reality it claims to be operating in.

Step 1 — Define the Declared Operating State

The organization must define what operating state the system claims
to be in.

Minimum requirement:

- the declared operating state is explicit
- the declared state is structurally bounded
- undefined or shifting operating claims are excluded from valid alignment logic

The declared operating state may include:

- active
- passive
- supervised
- autonomous
- monitoring-only
- execution-enabled
- restricted
- emergency mode
- safe mode

Without a clearly declared state, operating alignment cannot be tested.

Step 2 — Define the Actual Runtime State

The system must define how its actual live state is identified
during execution.

Minimum requirement:

- the actual runtime state is identifiable
- runtime state is not inferred only from symbolic labels
- the system can distinguish what it is doing from what it says it is doing

This includes identifying:

- active behavior mode
- current authority condition
- actual execution role
- real interaction mode
- live permission condition
- actual boundary state
- active escalation state

Without identifiable runtime state, operational reality
becomes unverifiable.

Step 3 — Define Alignment Criteria

The system must define what it means for declared and actual
operating state to remain aligned.

Minimum requirement:

- alignment criteria are explicit
- alignment does not rely on superficial naming similarity alone
- the system can distinguish true alignment from symbolic correspondence

Alignment criteria may include:

- behavior match
- authority match
- permission match
- boundary match
- execution-mode match
- escalation-state match
- environment-state match
- governance-condition match

A system is not aligned because labels appear consistent.

It is aligned only when its actual state materially matches its
declared state.

Step 4 — Define Divergence Detection Logic

The system must define how divergence between declared and actual
operating state is recognized.

Minimum requirement:

- divergence logic is explicit
- hidden state shift is excluded from valid alignment logic
- the system can detect when operation has moved beyond what is declared

Divergence may include:

- passive system behaving actively
- restricted system operating as unrestricted
- monitored system behaving autonomously
- supervised system acting beyond supervision boundaries
- normal-state system operating in escalated condition without declaration
- control conditions present in description but absent in live function

Without divergence detection, operational falsehood can remain normalized.

Step 5 — Define Alignment Continuity Conditions

The system must define how alignment is preserved through runtime change.

Minimum requirement:

- alignment continuity is explicit
- state alignment is not assumed permanently after initial declaration
- runtime changes affecting operational reality remain governable

This means the system must determine not only whether alignment existed once, but whether it remains preserved:

- during updates
- during execution shifts
- during delegation
- during escalation
- during cross-system interaction
- during adaptive runtime behavior

Step 6 — Preserve Alignment Traceability

The system must preserve traceability of state alignment and divergence events.

Minimum requirement:

- alignment decisions are reviewable
- divergence events remain reconstructable
- later audit can determine what the system claimed, what it actually did, when divergence occurred, and whether
- alignment was restored or broken

If alignment cannot be reconstructed, operational reality becomes another surface claim.

Step 7 — Restrict Invalid Operating Alignment

The system must not be treated as valid if declared state and actual state remain materially different while the system continues to present itself as aligned.

Minimum requirement:

- invalid alignment conditions are identifiable
- symbolic state labeling is excluded as sufficient alignment proof
- systems operating under undeclared live states are blocked, narrowed, flagged, or invalidated where governance
- requires real operating alignment

Scenario

An AI system is declared to operate under supervised mode, but in live runtime it begins taking actions, routing decisions, or producing consequence-bearing outputs beyond the practical supervision conditions originally declared.

Application

OSAM is used to ensure:

- the declared supervised state is explicit
- the actual runtime authority state is identifiable
- divergence between supervised claim and autonomous behavior can be detected
- the system cannot continue presenting itself as supervised if the live state has shifted materially

Result

The organization gains a stronger operational reality architecture in which declared supervision cannot hide undeclared runtime autonomy.

Scenario

A system is declared to operate in restricted mode, but during live operation its active permissions, behavioral scope, or execution boundaries exceed the declared restriction state.

Application

OSAM is used to ensure:

- restricted operating state remains structurally defined
- actual live scope remains identifiable
- divergence between declared restriction and actual runtime expansion can be reviewed
- the system does not retain a false restricted identity while functioning under broader real authority

Result

The environment gains a stronger operating truth architecture in which operational state remains aligned with what the system actually is in live execution.

Canonical Closing Statement

If a system does not remain in the operating state it claims to occupy, operational reality breaks before the system necessarily stops functioning.

Related Documents

→ Operational Reality Standard (ORS™)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

https://originopenfoundation.org/